# 添加用户态指令设计专题实验报告

**作者**：林岳　**学号**：2024K8009929018

**专题范围**：实践任务 10、11（在五级流水 CPU 上添加用户态运算指令，以及转移与字节/半字访存指令）

---

## 一、任务与判据

**任务 10（用户态运算指令）**：在五级流水（实践 7~9）基础上补齐立即数、比较、逻辑、移位与乘除法指令——`pcaddu12i`、`slti`、`sltui`、`andi`、`ori`、`xori`、`sll.w`、`srl.w`、`sra.w`、`mul.w`、`mulh.w`、`mulh.wu`、`div.w`、`mod.w`、`div.wu`、`mod.wu`；测试范围 **n1~n36**。

**任务 11（转移与访存扩展）**：新增有/无符号比较分支 `blt / bge / bltu / bgeu`，与字节/半字访存 `ld.b / ld.h / ld.bu / ld.hu / st.b / st.h`；测试范围 **n1~n46**。

验收判据沿用课程口径：xsim 逐指令比对参考执行流，全过打印 `----PASS!!!`；上板验收按板的成功显示判定。两版交付均附相对上一实验的逐文件 patch。

## 二、设计思路

### 2.1 乘法：ALU 内单周期实现

乘法作为 ALU 的普通运算（功能码 15/16/17），不占额外周期：ALU 内并行算两条 64 位积——无符号 `prod_u` 与有符号 `prod_s`；`mul.w` 取 `prod_u[31:0]`，`mulh.w` 取 `prod_s[63:32]`，`mulh.wu` 取 `prod_u[63:32]`。综合后映射到 DSP（占 8 个 DSP48E1）。

### 2.2 除法：独立 `divider` 模块（多周期）

`div.w / mod.w / div.wu / mod.wu` 由独立模块 `divider` 完成，顶层显式例化（接口见 §3.2）。有符号版本商向零截断、余数取被除数符号；`busy` 期间冻结前段、消费者在 ID 停等，结果经 EX→MEM→ID 前递——未完成值不会被送出去。

### 2.3 任务 11：分支与字节/半字访存

- **分支**：在 ID 判出（signed / unsigned 两套比较器），命中当拍冲刷取指前端、改写 PC，只付 2 个气泡；
- **字节/半字访存**：访存类型随流水线传递；写通路按地址生成字节掩码与重复数据，读通路按地址选择字节/半字并做符号/零扩展。

## 三、课上助教反馈与对应优化

### 3.1 助教提出的问题

课上助教就任务 11 的代码组织提出问题（关键词原样记录）：**接口、模块、实例化、定义重复、信号空置**。具体到代码：

1. **除法没有做成独立模块**——仍是顶层 EX 段里一段内联状态机（49 行），接口与实例化都不清晰；
2. **`alu_op[21:18]` 空置**——div/mod 的四位独热码在 ALU 内无人使用；

3. **定义重复**——div 判定有两处同义写法（`id_is_divmod` 与由 `alu_op` 高位或出的判据）；
4. **信号冗余**——除法相关信号组织松散。

### 3.2 优化一：除法器模块化 + 显式实例化

把内联状态机抽成独立模块并显式实例化，接口最小化、职责单一：

```
divider u_divider(
    .clk    (clk                      ),
    .reset  (reset                    ),
    .start  (ex_div_start             ),
    .unsign (inst_div_wu | inst_mod_wu ),
    .mod    (inst_mod_w  | inst_mod_wu ),
    .a      (id_rj_value              ),
    .b      (id_rkd_value             ),
    .busy   (ex_div_busy              ),
    .result (ex_div_result            )
    );
```

顶层只剩“发 start、看 busy、用 result”三件事。**等价性验证**：复跑 n1~n46 全 PASS，且仿真结束时间与改前**逐拍一致**（1419475 ns）——纯结构整改、行为零变化。

### 3.3 优化二：编码收窄与信号去重

- `alu_op` 由 **22 位收窄到 18 位**（删除 ALU 内部从不消费的 div/mod 四位）；
- div 判定只保留**单一定义**，经流水线寄存器携带到 EX 使用，删除重复写法；
- 全量“定义了但未使用”信号审计，清零。

### 3.4 优化三：除法算法提速

在模块化基础上换算法——**结果不变、周期数大降**：

- **平凡快速路径**：除数为 0、被除数为 0、`|被除数| < |除数|` 时，当拍直接出结果（原实现这些情形也要跑满 32 拍）；
- **对齐迭代**：先求前导零差 `shift = clz(b) − clz(a)`，把除数左移对齐被除数的最高位，只产生“有效商位”，迭代数 = `shift + 1`。例：`1000 ÷ 3` 只需 **9 拍**（商 333、余 1）；两个随机 32 位数相除常见 **1~4 拍**；最坏仍为 32 拍；
- **结构简化**：删除 64 位中间寄存器，符号修正移到输出组合完成。

**提速验证**：n1~n46 全 PASS；仿真结束时间 **1419475 → 1234675 ns（约 -13%）**，提速集中在除/模测试段（到测试点 n36 的累计时间 1125575 → 940775 ns）。

### 3.5 优化前后对照

| 项目 | 优化前 | 优化后（本版） |
|---|---|---|
| 除法器位置 | 顶层 EX 段内联状态机 | 独立 `divider` 模块 + 显式例化 |
| `alu_op` 宽度 | 22 位（含 4 位空置） | 18 位（无空置） |
| div 判定定义 | 两处同义写法 | 单一定义 + 寄存器携带 |
| 除法算法 | 固定 32 拍（恢复余数法） | 平凡快路径 + 对齐迭代（平均个位数拍） |

| | | |
|---|---|---|
| 关键寄存器 | 含 64 位中间寄存器 | 仅 4 个 32 位数据寄存器 |
| 仿真结束时间 | 1419475 ns | **1234675 ns（约 -13%）** |
| 功能正确性 | PASS | PASS（全 46 点） |

## 四、验证与结果

| 实验 | 测试范围 | 结果（xsim） |
|---|---|---|
| 任务 10 | n1~n36 | PASS，1126075 ns |
| 任务 11（优化前） | n1~n46 | PASS，1419475 ns |
| 任务 11（本版） | n1~n46 | **PASS，1234675 ns（约 -13%）** |

综合 / 实现：两个实验均 0 Error、时序收敛（布线后 WNS 分别为 +0.925 ns 与 +0.448 ns）；bit 已生成，上板验收待进行。

## 五、调试记录

1. **结构改动用“逐拍等价”验证**：模块化与去重后复跑，仿真结束时间分毫不变（1419475 ns）——无行为变更的强证据；算法优化版则“全点通过 + 时间下降”，证明只提速、不改结果。
2. **除法边角情形逐一核对**：除数 0、被除数 0、`|a| < |b|`、最小负数等，商/余数与旧版一致。

## 六、小结

1. 本专题在流水线上补齐乘除法与转移/访存类指令；乘法走 ALU、除法走独立多周期单元，是“面积 / 吞吐 / 可读性”折中的结果。
2. 按助教意见完成模块化与信号清理后，又对除法器做了算法级优化：**平均迭代从 32 拍降到个位数，仿真时间约 -13%。**
3. 两次改动都配了可复核的验证——结构改动“逐拍不变”，算法改动“全点通过 + 时间下降”。